



Network Programming – 05

afewhee@gmail.com



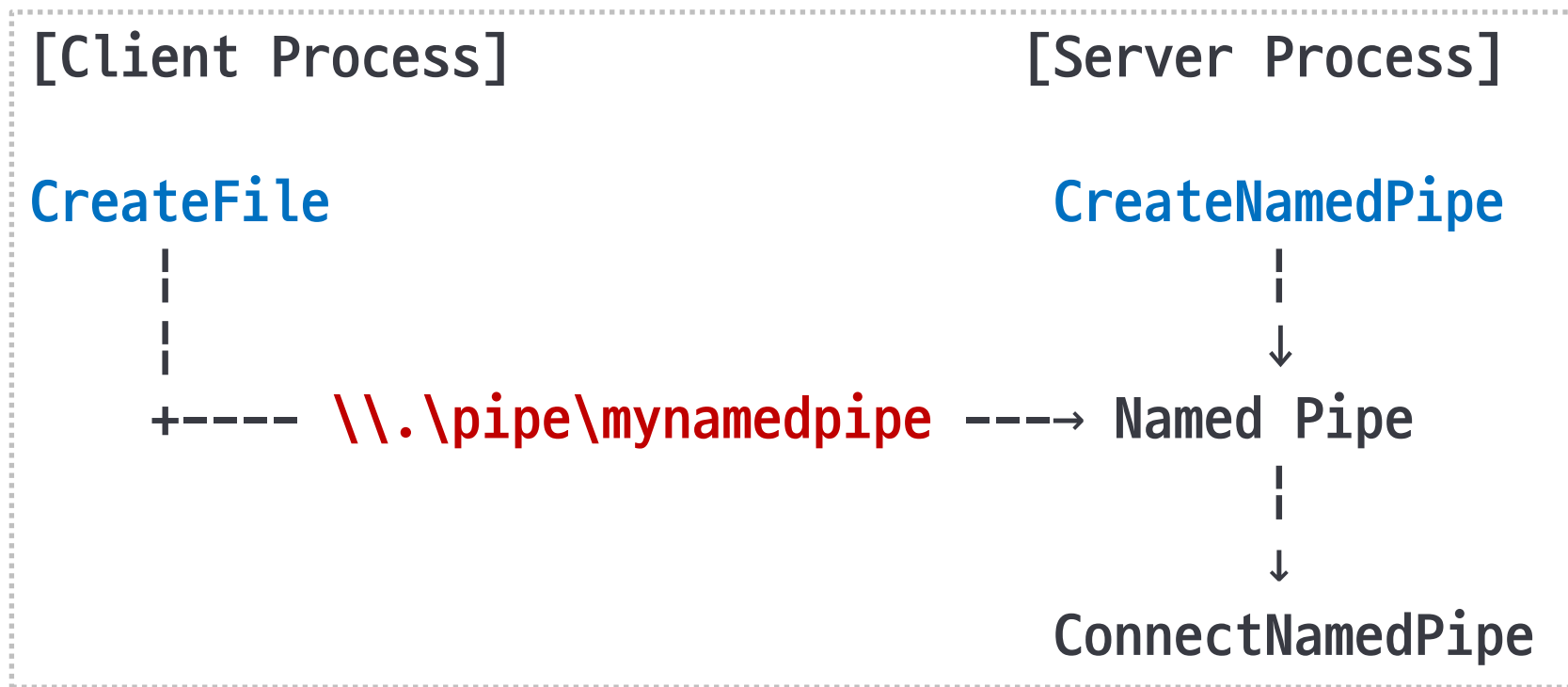


- Pipe
 - ◆ 프로세스 간 데이터 통로
 - ◆ 한쪽 Write
 - ◆ 다른 쪽 Read
- IPC
 - ◆ Inter-Process Communication
 - ◆ 프로세스 간 통신
- Named Pipe
 - ◆ 이름 있는 Pipe
 - ◆ 같은 PC 내부 프로세스 통신
 - ◆ 네트워크 경로 형식 사용 가능





- 서버 프로세스 클라이언트 프로세스 파이프 이름 기준 연결





● Pipe 소켓 비교

구분	파이프 (Pipe)	소켓 (Socket)
통신 범위	동일 호스트 내부 중심 Windows Named Pipe는 원격 통신 일부 지원	로컬 및 원격 네트워크
통신 방향	기본 단방향 Windows Named Pipe는 양방향 지원	기본 양방향
주소 체계	파일 시스템 경로 또는 파이프 네임스페이스 예: \\.\pipe\mypipe	IP 주소 + 포트 번호 Unix Domain Socket은 파일 경로 사용
오버헤드	낮음. 커널 버퍼 기반 프로세스 간 통신	상대적으로 높음. 네트워크 스택, 프로토콜 헤더 처리
주요 API	파일 I/O API: ReadFile, WriteFile	네트워크 API: send, recv, bind, listen
프로토콜	단순 바이트 스트림 또는 메시지	TCP, UDP 등 전송 계층 프로토콜





- 상황에 대한 소켓, Pipe 선택
 - ◆ 프로세스 간 통신 목적이 파이프. 같은 PC에서 유리

상황	적합 도
같은 PC 의 프로세스 간 통신	파이프
다른 PC와 통신	소켓
네트워크 확장 가능성 필요	소켓
단순 부모-자식 프로세스 통신	익명 파이프
서버/클라이언트 구조의 로컬 IPC	명명 파이프
TCP/UDP 기반 있음	소켓
파일 I/O 스타일 API	파이프
네트워크 연관	소켓





- Server
 - ◆ Named Pipe 생성
 - ◆ Client 접속 대기
 - ◆ Client 요청 수신
 - ◆ Server 응답 송신
- Client
 - ◆ Named Pipe 열기
 - ◆ Server로 문자열 송신
 - ◆ Server 응답 수신





<Client>

CreateFile

WriteFile

ReadFile

CloseHandle



<Server>

CreateNamedPipe

ReadFile

WriteFile

DisconnectNamedPipe





- 형식

- ◆ \\.\pipe\파이프이름

```
char g_sPipeName[] = "\\.\pipe\mynamedpipe";
```

- 의미

- ◆ `\\.` : 현재 컴퓨터
- ◆ `pipe` : Named Pipe 네임스페이스
- ◆ `mynamedpipe` : 사용자 지정 이름

- 주의

- ◆ Server와 Client 동일 이름 사용
- ◆ 문자열 이스케이프 때문에 `\\` 반복





\\.\pipe\mynamedpipe

		+- -	사용자 정의 이름
	+- - - - -		Named Pipe 경로
+- - - - -			로컬 컴퓨터





● PIPEINST

- ◆ OVERLAPPED :
비동기 I/O 정보
- ◆ hPipeInst :
Pipe 핸들
- ◆ chRequest :
Client 요청 버퍼
- ◆ cbRead :
수신 바이트 수
- ◆ chReply :
Server 응답 버퍼
- ◆ cbToWrite : 송신 바이트 수
- ◆ dwState : Pipe 상태
- ◆ fPendingIO : 대기 중 I/O 여부

```
typedef struct
{
    OVERLAPPED oOverlap;
    HANDLE hPipeInst;
    char chRequest[MAX_BUF];
    DWORD cbRead;
    char chReply[MAX_BUF];
    DWORD cbToWrite;
    DWORD dwState;
    int fPendingIO;
} PIPEINST;
```





● 상태 값 설정

```
#define CONNECTING_STATE 0  
#define READING_STATE 1  
#define WRITING_STATE 2
```

● 서버 상태 흐름

CONNECTING_STATE 클라이언트 연결 대기
READING_STATE 클라이언트 요청 수신 준비
WRITING_STATE 클라이언트 응답 송신 준비

CONNECTING_STATE



READING_STATE



WRITING_STATE



READING_STATE





- INSTANCES 파이프 인스턴스 개수
- PIPE_TIMEOUT 파이프 대기 시간
- MAX_BUF 송수신 버퍼 크기
- Pipe Instance Count
- Pipe[0], Pipe[1], Pipe[2], Pipe[3]

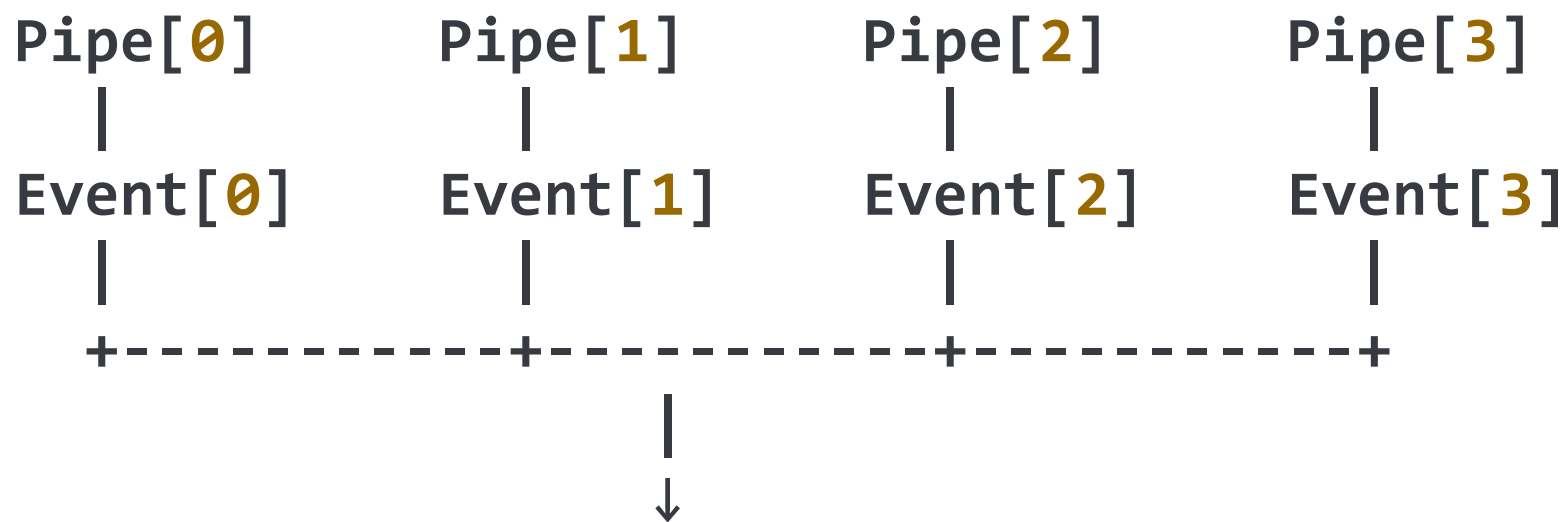
```
#define INSTANCES      4  
#define PIPE_TIMEOUT  5000  
#define MAX_BUF        8192
```





```
PIPEINST Pipe    [INSTANCES];  
HANDLE     hEvents[INSTANCES];
```

Pipe 배열 파이프 인스턴스별 상태 저장
hEvents 배열 Overlapped I/O 완료 이벤트



`WaitForMultipleObjects`





- Event
 - ◆ I/O 완료 알림 객체
 - ◆ WaitForMultipleObjects 대기 대상
- Overlapped I/O
 - ◆ 비동기 입출력
 - ◆ 요청 후 즉시 반환 가능
 - ◆ 완료 시 Event 신호 상태
 - `hEvents[i] = CreateEvent(nullptr, TRUE, TRUE, nullptr);`
 - `Pipe[i].oOverlap.hEvent = hEvents[i];`
- CreateEvent 인자
 - ◆ TRUE : 수동 리셋 이벤트
 - ◆ TRUE : 초기 신호 상태





```
hEvents[i] = CreateEvent(nullptr, TRUE, TRUE, nullptr);  
Pipe[i].oOverlap.hEvent = hEvents[i];
```

CreateEvent 이벤트 객체 생성

TRUE Manual-reset Event 수동 리셋 이벤트

TRUE Initial State Signaled 초기 신호 상태

OVERLAPPED.hEvent I/O 완료 통지 이벤트

CreateEvent



Manual-reset Event



OVERLAPPED.hEvent



I/O 완료 신호





9. CreateNamedPipe

```
hPipe = CreateNamedPipe(  
    g_sPipeName,  
    PIPE_ACCESS_DUPLEX | FILE_FLAG_OVERLAPPED,  
    PIPE_TYPE_MESSAGE | PIPE_READMODE_MESSAGE | PIPE_WAIT,  
    PIPE_UNLIMITED_INSTANCES,  
    MAX_BUF, MAX_BUF, NMPWAIT_USE_DEFAULT_WAIT,  
    nullptr);
```

PIPE_ACCESS_DUPLEX 양방향 통신

FILE_FLAG_OVERLAPPED Overlapped I/O 사용

PIPE_TYPE_MESSAGE 메시지 단위 파이프

PIPE_READMODE_MESSAGE 메시지 단위 읽기

PIPE_WAIT Blocking Mode

CreateNamedPipe

|

+-- Duplex

+-- Overlapped

+-- Message Type

+-- Message Read Mode

+-- Blocking Mode

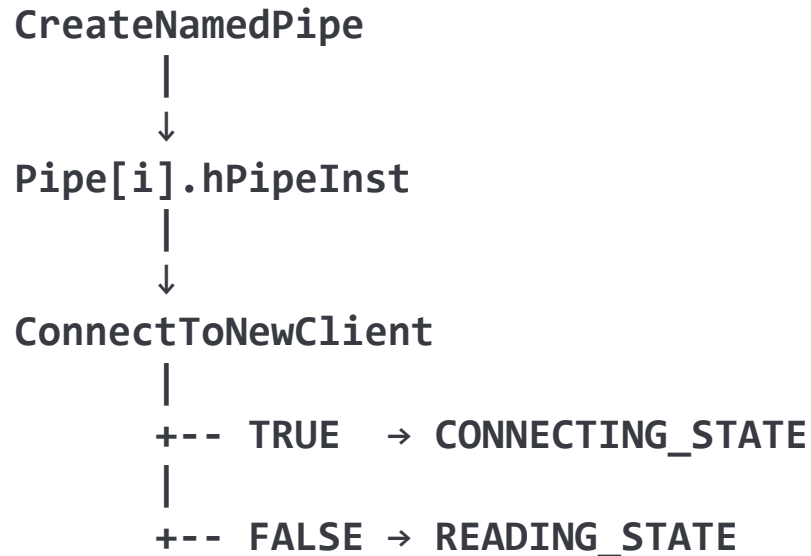




10. Pipe 인스턴스 초기화

```
Pipe[i].hPipeInst = hPipe;  
  
Pipe[i].fPendingIO =  
    ConnectToNewClient(Pipe[i].hPipeInst, &Pipe[i].oOverlap);  
  
Pipe[i].dwState = Pipe[i].fPendingIO ?  
    CONNECTING_STATE :  
    READING_STATE;
```

파이프 핸들 저장 클라이언트 연결 시작 미완료 상태 확인 초기 상태 결정





11. ConnectToNewClient

```
int ConnectToNewClient(HANDLE hPipe, LPOVERLAPPED lpo)
{
    int fConnected, fPendingIO = FALSE;
    lpo->Offset = 0;
    lpo->OffsetHigh = 0;

    fConnected = ConnectNamedPipe(hPipe, lpo);
}
```

ConnectNamedPipe 클라이언트 연결 요청 시작

OVERLAPPED 사용 즉시 완료 또는 대기 상태

Offset / OffsetHigh 파일 I/O용 위치 값 파이프에서는 0





11. ConnectToNewClient

ConnectToNewClient



ConnectNamedPipe



+ - - **ERROR_IO_PENDING**



fPendingIO = TRUE



+ - - **ERROR_PIPE_CONNECTED**



SetEvent





13. ERROR_PIPE_CONNECTED

```
case ERROR_PIPE_CONNECTED:  
    if(SetEvent(lpo->hEvent))  
        break;
```

ERROR_PIPE_CONNECTED 클라이언트 이미 연결됨
SetEvent 완료 이벤트 직접 신호 상태 설정

Client already connected



ERROR_PIPE_CONNECTED



SetEvent



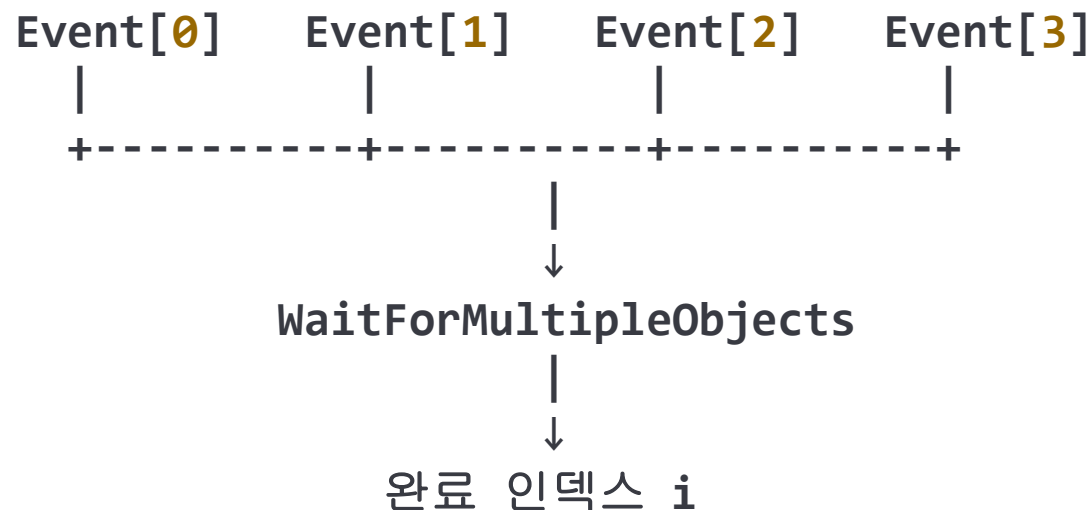
WaitForMultipleObjects 에서 감지





```
while(1)
{
    i = WaitForMultipleObjects(
        INSTANCES, hEvents, FALSE, INFINITE);
    i -= WAIT_OBJECT_0;
```

WaitForMultipleObjects 여러 이벤트 중 하나 대기
FALSE 하나만 신호 상태여도 반환
INFINITE 무한 대기
반환값 완료된 Pipe 인덱스 계산





15. Pending I/O 완료 확인

```
if(Pipe[i].fPendingIO) {  
    hr = GetOverlappedResult(  
        Pipe[i].hPipeInst,  
        &Pipe[i].oOverlap,  
        &cbRet,  
        FALSE);
```

fPendingIO 미완료 I/O 존재 여부

GetOverlappedResult Overlapped I/O 완료 결과 확인

cbRet 전송 완료 바이트 수

FALSE 추가 대기 없음

fPendingIO == TRUE

|

↓

GetOverlappedResult

|

+-- hr

|

+-- cbRet





```
switch(Pipe[i].dwState) {  
case CONNECTING_STATE:  
    Pipe[i].dwState = READING_STATE;  
    break;  
  
case READING_STATE:  
    Pipe[i].cbRead = cbRet;  
    Pipe[i].dwState = WRITING_STATE;  
    break;  
  
case WRITING_STATE:  
    Pipe[i].dwState = READING_STATE;  
    break;  
}
```





CONNECTING_STATE 완료 READING_STATE 전환
READING_STATE 완료 수신 바이트 저장 WRITING_STATE 전환
WRITING_STATE 완료 READING_STATE 전환

완료된 작업



상태 확인



+-- CONNECTING → READING



+-- READING → WRITING



+-- WRITING → READING





```
hr = ReadFile(  
    Pipe[i].hPipeInst,  
    Pipe[i].chRequest,  
    MAX_BUF * sizeof(char),  
    &Pipe[i].cbRead,  
    &Pipe[i].oOverlap);
```

ReadFile 클라이언트 요청 수신
chRequest 수신 버퍼
cbRead 수신 바이트 수
oOverlap Overlapped I/O 정보





READING_STATE



ReadFile



+ - - 즉시 완료



WRITING_STATE



+ - - ERROR_IO_PENDING



fPendingIO = TRUE





```
GetAnswerToRequest(&Pipe[i]);
```

```
hr = WriteFile(  
    Pipe[i].hPipeInst,  
    Pipe[i].chReply,  
    Pipe[i].cbToWrite,  
    &cbRet,  
    &Pipe[i].oOverlap);
```

GetAnswerToRequest 응답 데이터 준비
WriteFile 클라이언트 응답 송신
chReply 응답 버퍼
cbToWrite 송신 바이트 수





WRITING_STATE



GetAnswerToRequest



WriteFile



즉시 완료 → **READING_STATE**

ERROR_IO_PENDING → **fPendingIO = TRUE**





```
void GetAnswerToRequest(LPPIPEINST pipe) {  
    printf("[%d] %s\n", pipe->hPipeInst, pipe->chRequest);  
    strcpy(pipe->chReply, "Default answer from server");  
    pipe->cbToWrite = strlen(pipe->chReply);  
}
```

요청 문자열 출력 응답 문자열 복사 응답 크기 계산

chRequest



printf

"Default answer from server"



chReply



cbToWrite

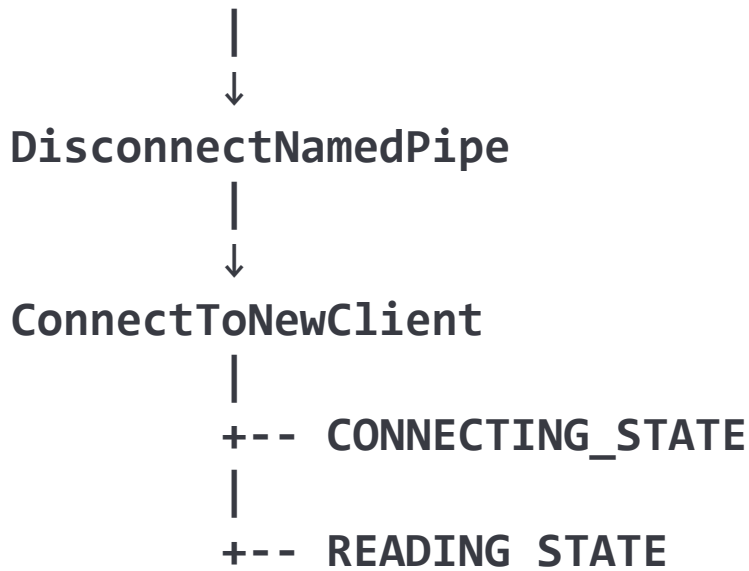




20. DisconnectAndReconnect

```
void DisconnectAndReconnect(DWORD i) {  
    DisconnectNamedPipe(Pipe[i].hPipeInst);  
    Pipe[i].fPendingIO =  
        ConnectToNewClient(Pipe[i].hPipeInst, &Pipe[i].oOverlap);  
    Pipe[i].dwState = Pipe[i].fPendingIO ?  
        CONNECTING_STATE : READING_STATE;  
}
```

클라이언트 연결 종료 같은 Pipe 인스턴스 재사용
새 클라이언트 연결 대기 상태 재설정
오류 또는 연결 종료





21. 클라이언트 CreateFile

```
g_hPipe = CreateFile(  
    g_sPipeName,  
    GENERIC_READ | GENERIC_WRITE,  
    0, nullptr, OPEN_EXISTING, 0, nullptr);
```

CreateFile Named Pipe 연결

GENERIC_READ 읽기 권한

GENERIC_WRITE 쓰기 권한

OPEN_EXISTING 이미 생성된 파이프 열기

Client



CreateFile



\\.\pipe\mynamedpipe



Server Pipe Instance





22. 클라이언트 접속 대기

```
if(g_hPipe != INVALID_HANDLE_VALUE)
    break;
hr = GetLastError();
if(ERROR_PIPE_BUSY != hr) {
    printf("Could not open pipe. GLE=%d\n", GetLastError());
    return -1;
}
hr = WaitForNamedPipe(g_sPipeName, 60000);
```

INVALID_HANDLE_VALUE CreateFile 실패
ERROR_PIPE_BUSY 모든 Pipe 인스턴스 사용 중
WaitNamedPipe Pipe 사용 가능 상태 대기

CreateFile

|
+-- 성공 → 연결 완료

|
+-- 실패



GetLastError



+-- ERROR_PIPE_BUSY → WaitForNamedPipe



+-- 기타 오류 → 종료





23. Message Read Mode 설정

```
dwMode = PIPE_READMODE_MESSAGE;  
hr = SetNamedPipeHandleState(g_hPipe, &dwMode, nullptr, nullptr);
```

PIPE_READMODE_MESSAGE 메시지 단위 읽기 모드

SetNamedPipeHandleState 클라이언트 Pipe 핸들 상태 변경

Pipe Handle



SetNamedPipeHandleState



PIPE_READMODE_MESSAGE



Message 단위 ReadFile

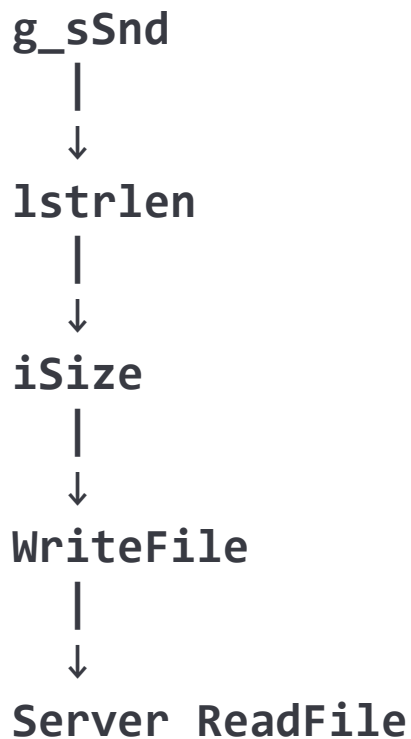




24. 클라이언트 WriteFile

```
iSize = strlen(g_sSnd);  
printf("Sending: %d byte\n", iSize);  
hr = WriteFile(g_hPipe, g_sSnd, iSize, &dTran, nullptr);
```

strlen 송신 문자열 길이 계산
WriteFile 서버로 메시지 송신
dTran 송신 완료 바이트 수





25. 클라이언트 ReadFile

```
do {  
    hr = ReadFile(g_hPipe, g_sRcv, MAX_BUF, &cbRead, nullptr);  
    if(!hr && GetLastError() != ERROR_MORE_DATA)  
        break;  
    printf("\"%s\"\\n", g_sRcv);  
} while(!hr);
```

ReadFile 서버 응답 수신

g_sRcv 수신 버퍼

ERROR_MORE_DATA 메시지 데이터 일부만 수신

반복 수신 남은 메시지 계속 읽기

Server WriteFile



Client ReadFile



+-- 전체 수신 → 출력

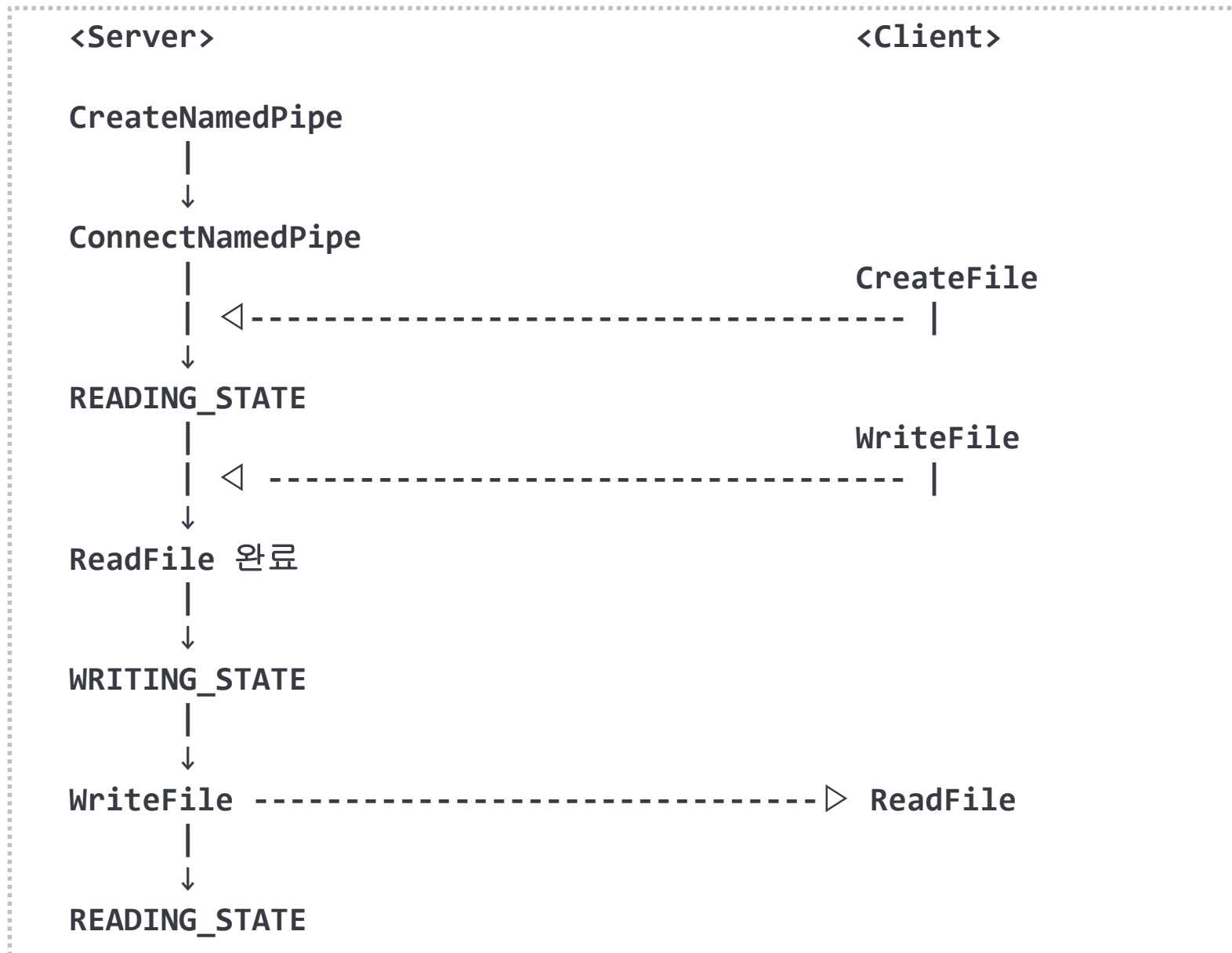


+-- ERROR_MORE_DATA



다시 ReadFile







27. Overlapped I/O 흐름

I/O 요청 시작



ReadFile / WriteFile / ConnectNamedPipe



+-- 즉시 완료



다음 상태 전환



+-- ERROR_IO_PENDING



Event Signal



WaitForMultipleObjects



GetOverlappedResult





Named Pipe 여러 인스턴스

인스턴스별 Event

인스턴스별 OVERLAPPED 상태 기반 흐름 제어

WaitForMultipleObjects 중심 대기

Pipe[0] -- Event[0] -- OVERLAPPED[0]

Pipe[1] -- Event[1] -- OVERLAPPED[1]

Pipe[2] -- Event[2] -- OVERLAPPED[2]

Pipe[3] -- Event[3] -- OVERLAPPED[3]



WaitForMultipleObjects



State Switch





CreateFile WaitNamedPipe

SetNamedPipeHandleState WriteFile ReadFile CloseHandle

CreateFile



WaitNamedPipe



SetNamedPipeHandleState



WriteFile



ReadFile



CloseHandle





- IPC
 - ◆ 로컬 프로세스 간 통신
 - ◆ Pipe Name 기반 연결
- I/O
 - ◆ ReadFile / WriteFile
 - ◆ Overlapped I/O
- Event
 - ◆ Manual-reset Event
 - ◆ WaitForMultipleObjects
 - ◆ GetOverlappedResult
- State
 - ◆ CONNECTING
 - ◆ READING
 - ◆ WRITING

